

Name: Jeese O M Flana
Reg No: 411625104021
Department: B.E CSE
Subject: Web Interface Unit 05
Batch : 04
Project 01 : To-Do Application

Aim

To design and develop a To-Do application using React Context API that allows users to add, complete, delete, and filter tasks using shared state management.

Procedure

1. Create a React application using Vite.
2. Create a TodoContext.jsx file to manage the application's global to-do state.
3. Use createContext() to create a Context for sharing to-do data between components.
4. Use useState() to store the list of to-do tasks and the selected filter.
5. Create an addTodo() function to add new tasks to the list.
6. Create a toggleTodo() function to mark tasks as completed or active.
7. Create a deleteTodo() function to remove tasks from the list.
8. Use useMemo() to display tasks according to the selected filter.
9. Create an AddTodo.jsx component to accept and add new tasks.
10. Create a TodoList.jsx component to display tasks and provide filtering options.
11. Create a TodoItem.jsx component to display individual tasks with checkbox and delete functionality.
12. Use useTodos() custom hook to access the shared Context data.
13. Wrap the application inside the TodoProvider component.
14. Create App.css to style the To-Do application.
15. Run the application and test adding, completing, deleting, and filtering tasks.

React Concepts Used

- **Context API** is used to share to-do data between components without passing props manually.
- **createContext()** is used to create the TodoContext.

- **useContext()** is used through the custom useTodos() hook to access shared data.
- **useState()** is used to manage the to-do list and selected filter.
- **useMemo()** is used to calculate and display filtered to-do items efficiently.
- **Functional Components** are used to create AddTodo, TodoList, TodoItem, and TodoProvider.
- **Props** are used to pass individual todo data to the TodoItem component.
- **Event Handling** is used for adding, completing, deleting, and filtering tasks.
- **Conditional Rendering** is used to display "Nothing here yet" when there are no tasks.
- **Array Methods** such as map(), filter(), and spread syntax are used to update and display the task list.
- **Custom Hook** useTodos() is created to simplify access to the Context.
- **Component-based Development** is used to divide the application into reusable components.
-

AddTodo.jsx

```
import { useState } from "react";
import { useTodos } from "../TodoContext";

export default function AddTodo() {
  const { addTodo } = useTodos();
  const [text, setText] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    addTodo(text);
    setText("");
  };

  return (
    <form onSubmit={handleSubmit} className="add-todo">
      <input
```

```

    type="text"
    value={text}
    onChange={(e) => setText(e.target.value)}
    placeholder="What needs doing?"
    aria-label="New todo text"
  />

  <button type="submit">
    Add
  </button>
</form>
);
}

```

TodoList.jsx

```

import { useTodos } from "../TodoContext";
import TodoItem from "../TodoItem";
export default function TodoList() {
  const {
    todos,
    filter,
    setFilter,
    activeCount,
    allCount,
  } = useTodos();
  return (
    <div className="todo-list">

      <div className="filters">

```

```

    {["all", "active", "done"].map((f) => (
      <button
        key={f}
        onClick={() => setFilter(f)}
        className={filter === f ? "active" : ""}
      >
        {f[0].toUpperCase() + f.slice(1)}
      </button>
    ))}
  </div>

  {todos.length === 0 ? (
    <p className="empty">
      Nothing here yet.
    </p>
  ) : (
    <ul>
      {todos.map((todo) => (
        <TodoItem
          key={todo.id}
          todo={todo}
        />
      ))}
    </ul>
  )}

  <p className="count">
    {activeCount} of {allCount} remaining
  </p>
</div>

);

```

```
}
```

TodoItem.jsx

```
import { useTodos } from "../TodoContext";

export default function TodoItem({ todo }) {

  const { toggleTodo, deleteTodo } = useTodos();

  return (

    <li className={`todo-item ${todo.done ? "done" : ""}`}>

      <label>

        <input

          type="checkbox"

          checked={todo.done}

          onChange={() => toggleTodo(todo.id)}

        />

        <span>{todo.text}</span>

      </label>

      <button

        onClick={() => deleteTodo(todo.id)}

        aria-label={`Delete "${todo.text}"`}

        className="delete-btn"

      >

        X

      </button>

    </li>

  );

}
```

TodoContext.jsx

```
import { createContext, useContext, useState, useMemo } from "react";
```

```

const TodoContext = createContext(null);

export function TodoProvider({ children }) {

  const [todos, setTodos] = useState([

    { id: 1, text: "Learn Context API", done: true },

    { id: 2, text: "Build a to-do app", done: false },

  ]);

  const [filter, setFilter] = useState("all");

  const addTodo = (text) => {

    if (!text.trim()) return;

    setTodos((prev) => [

      ...prev,

      {

        id: Date.now(),

        text: text.trim(),

        done: false,

      },

    ]);

  };

  const toggleTodo = (id) => {

    setTodos((prev) =>

      prev.map((t) =>

        t.id === id ? { ...t, done: !t.done } : t

      )

    );

  };

  const deleteTodo = (id) => {

    setTodos((prev) => prev.filter((t) => t.id !== id));

  };

  const visibleTodos = useMemo(() => {

```

```

    if (filter === "active") {
      return todos.filter((t) => !t.done);
    }
    if (filter === "done") {
      return todos.filter((t) => t.done);
    }
    return todos;
  }, [todos, filter]);
const value = {
  todos: visibleTodos,
  allCount: todos.length,
  activeCount: todos.filter((t) => !t.done).length,
  filter,
  setFilter,
  addTodo,
  toggleTodo,
  deleteTodo,
};
return (
  <TodoContext.Provider value={value}>
    {children}
  </TodoContext.Provider>
);
}

```

```

export function useTodos() {
  const ctx = useContext(TodoContext);
  if (!ctx) {
    throw new Error(

```

```
    "useTodos must be used inside a <TodoProvider>"
  );
}
return ctx;
}
```

App.jsx

```
import { TodoProvider } from "../TodoContext";
import AddTodo from "../AddTodo";
import TodoList from "../TodoList";
import "../App.css";
export default function App() {
  return (
    <TodoProvider>
      <div className="app">
        <h1>To-Do</h1>
        <AddTodo />
        <TodoList />
      </div>
    </TodoProvider>
  );
}
```

App.css

```
.app {
  max-width: 550px;
  margin: 50px auto;
  padding: 30px;
  font-family: "Segoe UI", sans-serif;
  background: #16251f;
```

```
    color: #f1f8f5;
    border-radius: 16px;
    box-shadow: 0 15px 40px rgba(0, 0, 0, 0.3);
}
```

```
.add-todo {
    display: flex;
    gap: 12px;
    margin-bottom: 25px;
}
```

```
.add-todo input {
    flex: 1;
    padding: 14px;
    border: 1px solid #496b5c;
    border-radius: 8px;
    background: #22372e;
    color: white;
    outline: none;
}
```

```
.add-todo input::placeholder {
    color: #9eb5aa;
}
```

```
.add-todo input:focus {
    border-color: #7ed6a8;
}
```

```
.add-todo button {
```

```
padding: 12px 20px;
border: none;
border-radius: 8px;
background: #7ed6a8;
color: #102019;
font-weight: bold;
cursor: pointer;
}

.add-todo button:hover {
    background: #a1e8c0;
}

.todo-item {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 16px;
    margin: 12px 0;
    background: #20352c;
    border-radius: 10px;
    border: 1px solid #355346;
    transition: 0.3s;
}

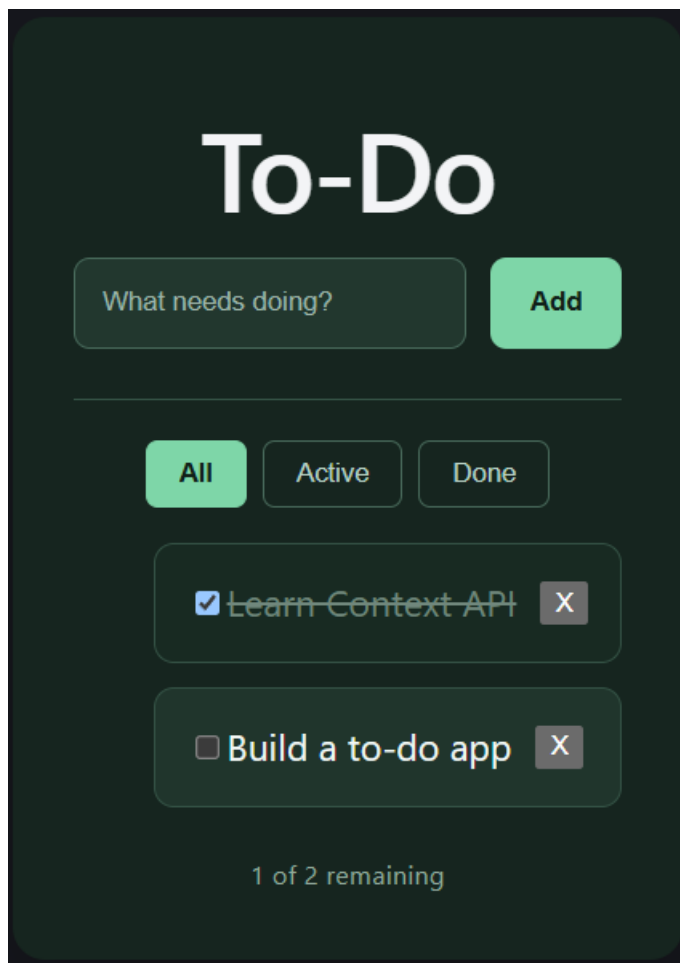
.todo-item:hover {
    transform: translateX(5px);
    border-color: #7ed6a8;
}

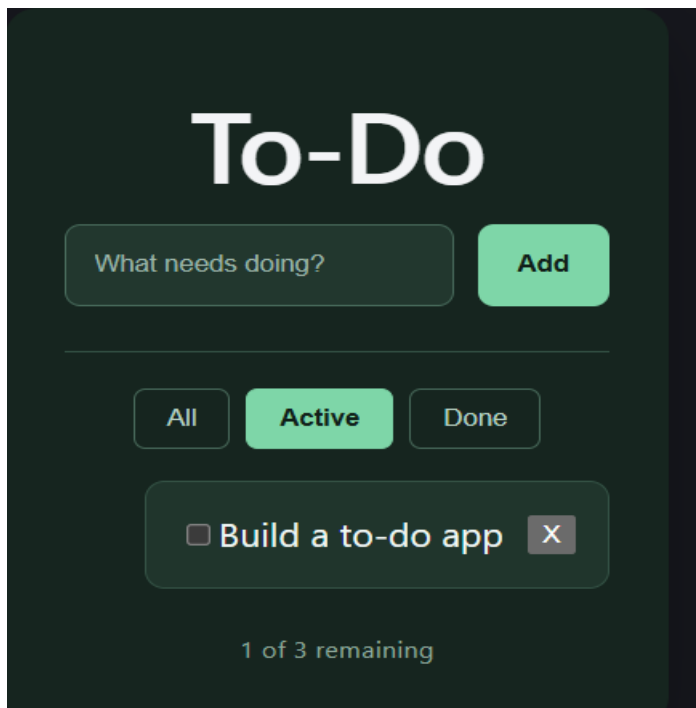
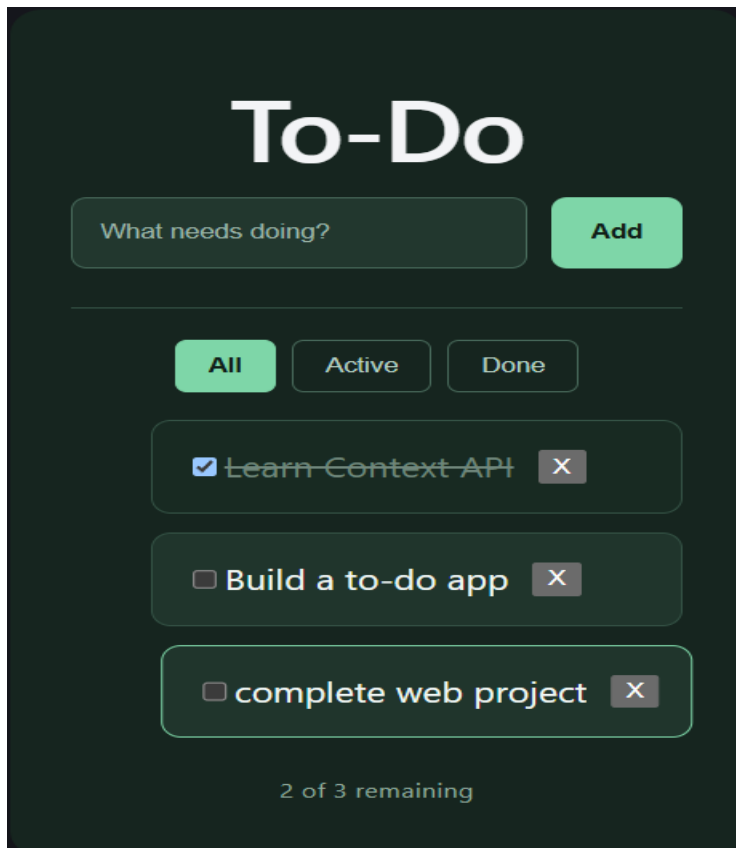
.todo-item span {
    flex: 1;
```

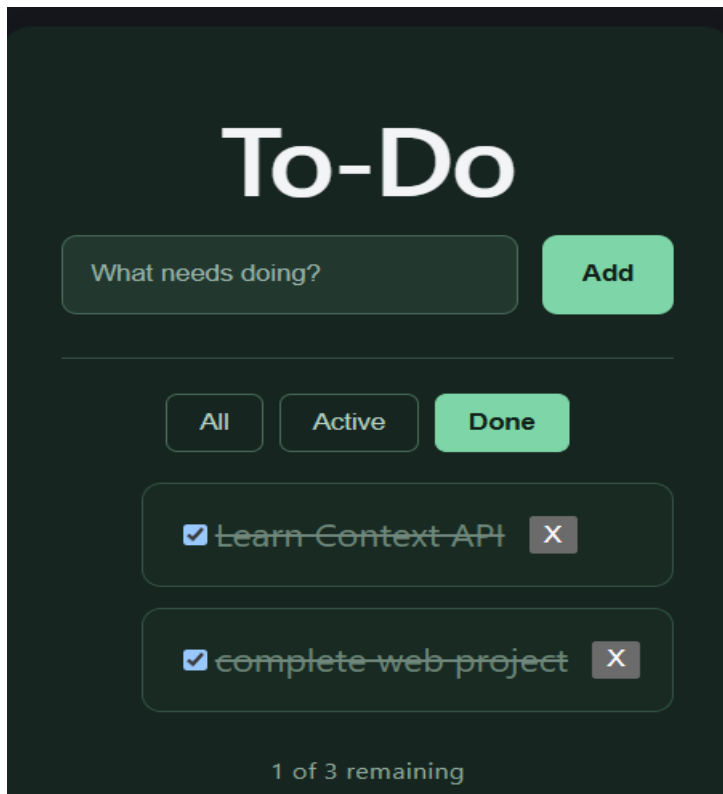
```
}  
  
.todo-item.done {  
    background: #182a22;  
}  
  
.todo-item.done span {  
    text-decoration: line-through;  
    color: #70877c;  
}  
  
.filters {  
    display: flex;  
    justify-content: center;  
    gap: 8px;  
    margin-top: 25px;  
    padding-top: 20px;  
    border-top: 1px solid #355346;  
}  
  
.filters button {  
    padding: 8px 16px;  
    border: 1px solid #496b5c;  
    border-radius: 6px;  
    background: transparent;  
    color: #b9d1c5;  
    cursor: pointer;  
}  
  
.filters button:hover {  
    background: #29483a;  
}
```

```
.filters button.active {  
    background: #7ed6a8;  
    color: #102019;  
    border-color: #7ed6a8;  
    font-weight: bold;  
}  
  
.count {  
    margin-top: 20px;  
    text-align: center;  
    font-size: 13px;  
    color: #91a99d;  
}
```

Output :







Result

The To-Do application was successfully developed using React. The project demonstrates the use of Context API for global state management, useState for managing tasks and filters, useMemo for filtering tasks, custom hooks for accessing shared state, reusable components, event handling, conditional rendering, and CSS styling. The application allows users to add, complete, delete, and filter tasks as All, Active, or Done.

